

ELEC 490/498 Final Report

LiteLM: A Pack-Based LLM for Portable On-Device Intelligence

Submitted by Group #20

Group Members:

Luka Rodrigues (20291000) - (20lfr@queensu.ca)
Vivek Chokkalingam (20285333) - (20vdc1@queensu.ca)
Tristan Alderson (20222127) - (19tja3@queensu.ca)
Andy Wang (20287448) - (20yw175@queensu.ca)

Faculty Supervisor: Ryan Grant, reg1@queensu.ca

Date: April 7th, 2026

Executive Summary

LiteLM is an edge AI system that runs a quantized GPT-2 language model entirely offline on the Xilinx Kria KV260 FPGA development board. The device accepts natural language queries and generates text responses without any network connection, targeting users in environments where cloud-based AI is unavailable: remote classrooms, field medical stations, and rural workshops. The project is motivated by the fact that approximately 2.2 billion people remain offline globally, and cloud-dependent AI tools fail in exactly the connectivity-constrained settings where portable information access would be most useful.

The system is built on the Zynq UltraScale+ MPSoC, which integrates a quad-core ARM Cortex-A53 processor and FPGA programmable logic on a single chip. The hardware accelerator consists of four modules developed in Vitis HLS: a Scheduler FSM that sequences all inference operations, a Memory Management Unit that stages weight and activation data between DDR4 and on-chip URAM, a Compute Controller that executes tiled INT8 matrix-vector multiplications, and AXI communication interfaces that connect the FPGA fabric to the ARM processor and external memory. The ARM processor handles tokenization, embedding lookup, and non-linear operations (layer normalization, softmax, GELU) in floating point, while the FPGA accelerates the matrix multiplications that dominate transformer inference compute.

The original design targeted Phi-3 Mini with INT4 quantization and full on-FPGA inference including non-linear activations. Three factors forced revisions during development. INT4 quantization produced unusable model outputs, requiring a shift to INT8. Phi-3 Mini's memory footprint at INT8 (~1.77 GB) created DDR pressure under Linux, and its activation functions (RoPE, SwiGLU) demanded FPGA resources the design could not spare. The team switched to GPT-2, which at ~0.53 GB post-INT8 fits comfortably in memory and uses simpler activations. In March 2026, after integration testing revealed timing and handshake failures at full scale, the team executed a planned fallback: non-linear operations moved to the CPU, and the FPGA role narrowed to MAC-only computation. This pivot preserved the core contribution, FPGA-accelerated matrix computation for transformer inference, while producing a working end-to-end demonstration.

The accelerator was verified through HLS C-simulation with deterministic test vectors at reduced dimensions, quantization pipeline validation against FP32 baselines, and PS-PL integration testing through both mock and on-board environments. The total hardware cost was approximately \$475 CAD, well within the \$600 CAD budget cap. The KV260 draws roughly 3.6 W under inference workload, compared to 250-400 W for a cloud GPU, making edge deployment practical for power-constrained settings.

The project demonstrates that FPGA-accelerated LLM inference at the edge is feasible on a student budget, though the gap between module-level simulation and full-scale hardware integration proved to be the largest underestimation in the original plan.

Table of Contents

EXECUTIVE SUMMARY	1
1. PROBLEM SCOPE AND DEFINITION	1
2. DESIGN PROCESS OVERALL	3
2.1 KEY DESIGN DECISIONS (OCTOBER 2025 – MARCH 2026).....	4
2.1.1 <i>Decision 1 - Development Board: Kria KV260 (October 2025)</i>	4
2.1.2 <i>Decision 2 - Compute Strategy: Tiling over Systolic Array (October 2025)</i>	5
2.1.3 <i>Decision 3 - Weight Quantization: FP32 → INT4 → INT8 (October 2025)</i>	5
2.1.4 <i>Decision 4 - Model Architecture: Phi-3 Mini → GPT-2 (March 2026)</i>	6
2.2 SYSTEM ARCHITECTURE (OCTOBER - NOVEMBER 2025)	8
2.2.1 <i>Hardware (PL)</i>	9
2.2.2 <i>Software/Firmware (PS) Phi3 only</i>	11
3. IMPLEMENTATION	13
3.1 HARDWARE (PL)	13
3.1.1 <i>Top-level and Shared Params</i>	13
3.1.2 <i>Scheduler FSM</i>	14
3.1.3 <i>Memory Management Unit (MMU)</i>	14
3.1.4 <i>Compute Controller (CC)</i>	16
3.1.5 <i>AXI Communication</i>	17
3.1.6 <i>Top Module</i>	18
3.1.7 <i>Error Handling</i>	18
3.2 SOFTWARE/FIRMWARE (PS)	19
3.2.1 <i>The LLM Model</i>	19
3.2.2 <i>Firmware</i>	20
4. PROJECT COMPLIANCE WITH TARGET SPECIFICATIONS	22
5. SSEERP FACTORS	23
5.1 SAFETY.....	23
5.2 SOCIAL	24
5.3 ENVIRONMENTAL	24
5.4 ECONOMIC	25
5.5 REGULATORY	25
5.6 PROFESSIONAL	26
6. TESTING / EVALUATION	26
6.1 PL DATAPATH VERIFICATION	26
6.2 QUANTIZATION PIPELINE VERIFICATION	27
6.3 MEMORY BUDGET ANALYSIS	28
6.4 INTEGRATION TESTING	28
6.5 PERFORMANCE COMPARISON	28
6.6 DESIGN ITERATION DRIVEN BY TESTING	29
7. APPLIES AND EVALUATES TOOLS.....	29
8. PROJECT PLANNING AND BUDGETING.....	30
8.1 ORIGINAL PLAN AND TIMELINE	31
8.2 EXECUTED PLAN	31
8.3 THE MARCH PIVOT	32
8.4 BUDGET	33
8.5 CRITICAL PATH AND DEPENDENCIES	33
9. PROJECT REFLECTIONS	34

9.1 TEAM-LEVEL REFLECTION..... 34

9.2 BROADER LESSONS 35

REFERENCES36

APPENDIX37

GLOSSARY 37

RELEVANT FIGURE 39

MEMBER CONTRIBUTIONS 41

ACKNOWLEDGMENTS..... 41

Table of Tables

Table 1: Project design process timeline.4

Table 2: Phi-3 Mini vs. GPT-2 architectural comparison for hardware inference8

Table 3: DmaSel - DMA fetch opcodes issued by the Scheduler FSM to the MMU.....15

Table 4: MMU internal memory structures16

Table 5: Compute Op - compute dispatch opcodes issued by the Scheduler FSM to the Compute Controller16

Table 6: transformer_top - external port definitions.....18

Table 7: Summary of functional, interface, and performance requirements validation for the final system build. .22

Table 8: Summary of tools used to design, implement, and integrate the system.29

Table of Figures

Figure 1: Complete system block diagram showing all module-level signal connections between the Scheduler FSM, MMU, Compute Controller, AXI, and DDR memory.....39

Figure 2: Gantt chart timeline from Oct 5th 2025 to March 30th 2026.....40

1. Problem Scope and Definition

Modern large language models depend on cloud infrastructure for inference. A user types a query, the query travels to a remote data center, a GPU cluster runs the model, and the response returns over the network. This workflow assumes two things, a stable internet connection and willingness to send private data to a third party. Both assumptions fail in exactly the contexts where AI-assisted information would be most useful.

The scale of this failure is not hypothetical. According to the International Telecommunication Union, approximately 2.2 billion people remain offline as of 2025, roughly one quarter of the global population [1]. Nearly 96% of those individuals live in low- and middle-income economies. Even where connectivity exists, the divide between urban and rural access is steep, with 85% of urban populations use the internet compared to 58% of rural populations [1]. Mobile broadband remains unaffordable in roughly 60% of low- and middle-income markets. And even in high-income countries, connectivity gaps persist in wilderness areas, marine environments, and remote industrial sites where cloud-dependent tools become unavailable at precisely the moments they could matter most.

LiteLM responds to this gap. The project originated in November 2024 with the goal of accelerating LLM inference on an integrated FPGA and embedded CPU system using a small, constrained model. The motivation is that since cloud-scale AI inference requires expensive, power-hungry hardware, how far can a quantized transformer model could be pushed on a cheap FPGA dev board, how small the model could go while still producing useful outputs. From there, the search began for development boards that fit a student budget and had both FPGA fabric and an integrated CPU on the same chip. The result was the Xilinx Kria KV260, a single-board platform combining a quad-core ARM Cortex-A53 processor with Zynq UltraScale+ programmable logic. The device accepts a text query, processes it through a hardware-accelerated transformer pipeline, and returns a generated response without any network connection.

The team studied two transformer architectures during the design phase. The GPT-2 [2] documentation provided the model dimensions, attention structure, and feed-forward network configuration that formed the basis of the hardware design, specifically the 12-layer, 768-dimensional architecture with 12 attention heads and GELU activation. The Phi-3 Mini [3] documentation was reviewed as the original target architecture, covering its RoPE

positional encoding scheme and SwiGLU activation in the feed-forward network. Both references were used when defining the sequence of transformer operations the Scheduler FSM and Compute Controller would need to execute, and when specifying the opcode set and tile indexing format shared across all hardware modules. Before committing to a compute architecture, the team reviewed the literature on FPGA-based transformer inference. The most relevant paper was "Language Model Inference on FPGA with Integer-only Operations" [4], which covered two things directly applicable to this design, how to handle non-linear activations (softmax, layer norm, GELU) using integer-only arithmetic on FPGA, and a systolic array implementation that streams model parameters through the compute fabric. The paper's systolic array approach was evaluated and ultimately not adopted, not because it was wrong, but because of debug constraints on the project's timeline. That decision is covered in Section 2.3, Decision 4.

At the hardware level, LLM inference reduces to a repeated cycle through the transformer block, projecting inputs into Query, Key, and Value matrices for self-attention, then passing the result through a feed-forward network. In GPT-2, this cycle iterates across 12 layers, 12 attention heads, and a 3072-dimensional intermediate network before producing an output from a 50,257-token vocabulary. Nearly all of these operations are matrix-vector multiplications. LiteLM focuses on this computational bottleneck, accelerating those specific matrix operations in the FPGA fabric to enable the inference cycle to run disconnected from the cloud.

Three use-case scenarios motivated the design. A backcountry medic with no cell signal could query symptom triage protocols. A mechanic in a remote community could look up vehicle repair procedures for a specific model. A teacher in an under-resourced school could use the device as an interactive tutor for students studying math, trigonometry, or chemistry.

The third scenario connects directly to United Nations Sustainable Development Goal 4, which calls for inclusive and equitable quality education and lifelong learning opportunities for all [5]. In the least developed countries, over two thirds of primary schools lack access to digital tools. The COVID-19 pandemic demonstrated the consequences of this gap, students without digital access experienced learning losses that the Brookings Institution documented as severe, and in many cases, lasting [6]. LiteLM addresses this by providing offline, domain-specific tutoring from a portable kit.

The hardware platform imposes hard constraints that shaped every design decision. The KV260 provides 4 GB of DDR4 system memory, approximately 18.4 Mb of URAM, and 5.1 Mb of BRAM on-chip. The project budget was capped at \$600 CAD. Performance targets included prompt-to-first-token latency under 300 ms and token generation above 20 tokens per second. These constraints required the team to quantize the model to INT8 precision, adopt a tiled matrix-vector compute strategy over a systolic array approach, and ultimately switch from Phi-3 Mini to GPT-2 when the larger model's memory footprint and activation complexity proved infeasible on the target hardware. Each of these decisions is documented in Section 2.3.

The stakeholders for this system include end users in connectivity-constrained settings (educators, field medics, technicians), the development team, the supervising faculty, and, in a future deployment context, any organization distributing the device. The primary constraint is hardware resource availability. The primary priority is producing a working end-to-end inference demonstration that validates the feasibility of FPGA-accelerated LLM inference at the edge.

Standard engineering design elements in this project include AXI4 protocol-compliant interfaces (AXI-Lite, AXI-Stream, AXI-Full) conforming to the AMBA 4 specification, fixed-point arithmetic in the compute pipeline, and finite state machine control logic. Non-standard elements include the tiled weight-staging strategy through the Memory Management Unit, the custom opcode-driven dispatch interface between the Scheduler FSM and Compute Controller, and the decision to implement all transformer layer sequencing in a single centralized FSM rather than distributing control across modules. These architectural choices, described in Section 2.4, were tailored to the specific debugging and resource constraints of this project rather than drawn from an established reference design.

2. Design Process Overall

This section describes the end-to-end design process followed throughout the project, from initial concept through hardware implementation. A tiled execution model was adopted across four hardware modules and two software components: the Scheduler FSM, which sequences all DMA and compute operations; the Memory Management Unit (MMU), which stages weight and activation data between DDR4 and on-chip URAM; the Compute Controller (CC), which executes the tiled INT8 matrix-vector multiplications; the Top Module, which integrates

all hardware into a single AXI-connected accelerator; and the LLM Model and Firmware, which handle weight quantization and PS-side inference control respectively.

The project was structured across four major phases, each building on the decisions made in the previous one.

Table 1 summarizes the timeline.

Table 1: Project design process timeline.

Phase	Key Tasks	Start	End
Idea & Scope	Board selection, quantization strategy, literature review	Nov 2024	Sep 2025
Architecture Design	Block diagram, FSM design, MMU/CC interface definition, opcode specification	Oct 2025	Nov 2025
Implementation	Scheduler FSM, MMU, Compute Controller, Top Module (Luka); AXI Communication (Tristan); Software model & firmware (Andy)	Dec 2025	Feb 2026
Integration & Testing	HLS simulation (csim/cosim), hardware bring-up, debug	Feb 2026	March 2026

The timeline was intentionally front-loaded with design decisions. The board, quantization strategy, and model architecture were all locked in before implementation began, following the principle that late-stage architectural changes to HLS designs are extremely costly to debug. Extra schedule margin was built into the implementation phase to account for iterative HLS simulation and the inherent complexity of hardware-software co-design.

2.1 Key Design Decisions (October 2025 – March 2026)

The following decisions were made before and during the design process. Each decision describes the reasoning behind the choice, the impact on the project, and where applicable, how the decision changed during development.

2.1.1 Decision 1 - Development Board: Kria KV260 (October 2025)

Two hard constraints drove the board selection. First, the board had to combine FPGA fabric and an embedded CPU on the same chip, the CPU runs model firmware and handles OS-level tasks, while the FPGA fabric runs the accelerator. Discrete FPGA and CPU on separate chips was ruled out because inter-chip PCIE communication adds latency and integration complexity. Second, the project budget was capped at \$600 CAD.

The Kria KV260 hit both at \$450 CAD. It runs a Zynq UltraScale+ MPSoC [7], providing a quad-core ARM Cortex-A53 Processing System (PS) and programmable logic (PL) on the same board, connected by high-bandwidth AXI interconnects [8]. The board also provides 4 GB of DDR4 for the PS and direct memory access from the PL, on-chip URAM and BRAM for weight staging, and a well-documented AXI protocol on the PL side.

The tight PS/PL integration via AXI is exactly the communication model this design needed for the firmware-to-hardware interface.

The KV260 set the constraints for everything downstream. Available URAM and BRAM defined the tile sizes. The 4 GB DDR4, shared with Linux, defined the model size budget. The PL-side AXI interface defined the external port structure of the top module. This decision held throughout the entire project and was never revised.

2.1.2 Decision 2 - Compute Strategy: Tiling over Systolic Array (October 2025)

Two approaches were considered for structuring matrix-vector computations in the Compute Controller. A systolic array, as used in the integer-only inference paper, streams weight data and input activations through a grid of processing elements simultaneously, performing MAC operations while data is in flight. This is efficient: high throughput, strong data reuse, well-suited for a bandwidth-constrained system. But debugging a systolic implementation is significantly harder. A single timing error or handshake glitch anywhere in the stream can corrupt the entire computation, and because everything is in flight at once, isolating the fault in simulation is slow. Tiling was chosen instead. Data is transferred from DDR into on-chip URAM, in statically sized chunks. All arithmetic for a given tile completes before the next tile is fetched. It is sequential and lower in throughput than systolic, that tradeoff was understood going in. The reason tiling was selected is how it fails: when something breaks in a tiled design, the fault is localized to one tile operation. One Scheduler FSM state, one Memory Management Unit (MMU) fetch, one Computer Controller (CC) computation. Each step can be verified independently in csim before moving on. Debug turnaround is fast, and fast debug turnaround was more valuable than peak throughput on this project timeline.

Tiling shaped the entire control architecture. The Scheduler FSM issues sequential DMA and compute requests one tile at a time. The MMU stages data in URAM banks between fetches. The CC processes one tile per dispatch. Every handshake, opcode, and state transition in the design reflects the tiled execution model. This decision held throughout the project; it was never revised. A systolic implementation would be worth revisiting in a future version with more development time, but for this design, tiling was the right call.

2.1.3 Decision 3 - Weight Quantization: FP32 $\rightarrow$ INT4 $\rightarrow$ INT8 (October 2025)

Weight quantization was fundamentally a memory problem. The KV260 has limited on-chip URAM/BRAM, and DDR4 bandwidth is a bottleneck when streaming large weight matrices into the compute units during inference.

Smaller bit-widths mean more weights fit on chip, fewer DDR4 fetches per inference step, and simpler MAC units. Running FP32 weights, the native training format, on FPGA fabric requires dedicated floating-point units that consume LUTs and DSPs at a scale that isn't practical for this design. INT4 quantization (4-bit) was the first target. The appeal was straightforward: half the storage of INT8, meaning roughly twice the weights in the same memory footprint. In practice, INT4 produced unusable outputs. The representational precision collapses at 4 bits, the model can no longer distinguish between nearby weight values, and output quality degraded to the point of being meaningless. The results made this clear quickly, and INT4 was dropped. INT8 quantization (8-bit) was the final choice (made in February 2026). Output quality at INT8 was acceptable while still cutting weight storage to 1 byte per value. All learned weight matrices: Q, K, V projections, output projection, FFN weights, and the logit projection, are stored as INT8.

The switch from INT4 to INT8 doubled the per-layer memory footprint, which directly fed into the model architecture decision that followed. INT8 quantization defined the tile geometry, the DMA transfer sizes, and the URAM bank layout for the entire design.

2.1.4 Decision 4 - Model Architecture: Phi-3 Mini → GPT-2 (March 2026)

The initial target architecture was Phi-3 Mini. Phi-3 Mini is a knowledge-distilled model, trained to perform well on a compressed knowledge set, which means it retains output quality better than most models of similar size after quantization. For a post-INT8 deployment on constrained hardware, that property was directly relevant. Its self-attention uses Rotary Position Embedding (RoPE) for positional encoding and its FFN uses SwiGLU activation, both of which are considered improvements over older approaches in terms of output quality. Two problems made Phi-3 Mini infeasible on the kria 260, hardware complexity of activation functions and DDR memory pressure.

Hardware complexity of RoPE and SwiGLU in the Phi3 Architecture

Both features require operations that are expensive to implement in FPGA logic with the available LUT budget. RoPE encodes token position by rotating the Q and K attention vectors using sinusoidal functions evaluated at runtime for each sequence position and attention head:

$$i = \text{current vector index}$$

$$m = \text{current token index}$$

$$d_{head} = \text{head dimension}$$

$$\theta[i] = \frac{1}{10,000^{2i/d_head}}$$

Equation 1: RoPE variable descriptions

$$\begin{aligned} q'[2i] &= q[2i] * \cos(m\theta[2i]) - q[2i + 1] * \sin(m\theta[2i]) \\ q'[2i + 1] &= q[2i] * \sin(m\theta[2i]) + q[2i + 1] * \cos(m\theta[2i]) \end{aligned}$$

Equation 2: RoPE rotation equations for current output element in Phi3 architecture

Implementing cos and sin on FPGA either requires a CORDIC unit per attention head (expensive in LUTs and DSPs) or a large lookup table, which consumes BRAM. On the xck26, neither was affordable given that those same resources were needed for the MAC arrays in the Compute Controller.

SwiGLU compounds the problem further. It requires two separate full weight matrices (W_{gate} and W_{up}) and a sigmoid non-linearity:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Equation 3: Sigmoid Function

$$FFN(x) = \{[(W_{gate} * x) \odot \sigma(W_{gate} * x)] * (W_{up} * x)\} * W_2$$

Equation 4: SwiGLU Feed Forward Network Equation for Phi3 Architecture

Doubling the FFN weight matrices directly doubles the per-layer memory footprint for the FFN stage. The sigmoid function, like RoPE's sin/cos, is a non-linear operation that doesn't map cleanly to integer-only hardware without a lookup table or dedicated compute unit (both of which cost resources that weren't available).

DDR4 memory pressure of the Phi3 Architecture

Post-INT4, Phi-3 Mini has a weight footprint of approximately **1.77 GB**. While that fits within the KV260's 4 GB DDR4, the Linux kernel running on the ARM PS actively manages that memory. The OS allocates DDR4 for page cache, kernel buffers, and runtime libraries. In practice, getting 1.77 GB of contiguous, reliably accessible memory for model weights (alongside everything Linux is doing) produced memory pressure issues. The firmware would load weights into DDR and the kernel would reclaim pages under pressure, causing inference failures and corrupted outputs.

The architecture was switched to GPT-2 because it resolves all three of the above problems cleanly.

- **No RoPE:** GPT-2 uses learned absolute positional embeddings, a simple table lookup indexed by token position. No sin, no cos, no CORDIC units.
- **GELU instead of SwiGLU:** Single weight matrix W_1 and one activation function approximable with a compact integer lookup table.
- **~0.53 GB post-INT8:** Fits comfortably in DDR4 with clear headroom for the OS and firmware. No memory pressure issues.

Table 2: Phi-3 Mini vs. GPT-2 architectural comparison for hardware inference

Attribute	Phi-3 Mini (original)	GPT-2 (final)
Post-INT size	~1.77 GB (INT4)	~0.53 GB (INT8)
Positional encoding	RoPE: sin/cos at runtime per (position, head)	Learned embeddings: table lookup only
FFN activation	SwiGLU: sigmoid + doubled weight matrices (W_{gate} , W_{up})	GELU: single W_1 , lookup-table approximable
DDR feasibility	Marginal: Linux kernel competes for DDR bandwidth	Comfortable: headroom for OS + firmware
HW complexity	High: CORDIC units, sigmoid, large tile sizes	Low: no trig, simpler FFN, smaller tiles

Switching to GPT-2 simplified almost every downstream hardware decision; smaller tiles, simpler activation functions, no trig units, and a DDR4 footprint the OS doesn't fight over. The tradeoff is that GPT-2's output quality is lower than Phi-3 Mini's; it was trained on an older, smaller dataset with no instruction tuning. For a hardware feasibility project on a constrained FPGA, that tradeoff was worth making. This was the most significant architectural revision in the project. Once made, the GPT-2 target held through to implementation.

2.2 System Architecture (October - November 2025)

With all four design decisions locked in, the block-level architecture was defined. The Phi-3 Mini and GPT-2 architecture documentation was used as reference for mapping the transformer operation sequence onto hardware modules. The system was partitioned into four subsystems:

- **AXI Communication:** All communication between the ARM firmware and FPGA fabric via the AMBA AXI protocol [9]. AXI4-Lite for control and status register access, AXI4-Stream for PS solicited high-throughput data ingress/egress, and AXI4-Full for PL solicited high throughput data ingress/egress.
- **Scheduler FSM:** Central control unit of the entire accelerator. Tracks transformer progression through all layers and operations and orchestrates timing and handshaking between all internal modules.

- **Memory Management Unit (MMU):** Manages all on-chip URAM banks and services DMA requests from the Scheduler FSM. Handles data staging between DDR4, URAM, and the Compute Controller.

Compute Controller (CC): Executes all matrix-vector multiplications in the transformer pipeline using INT8 weights and INT32 accumulators, operating on tiled data buffers prepared by the MMU. Figure 1 (In the Appendix) shows the full system block diagram, illustrating how all modules connect and communicate.

2.2.1 Hardware (PL)

AXI-Communication

The AXI (Advanced eXtensible Interface) facilitate control/status signaling and high-bandwidth data transfer between the Processing System (PS) and Programmable Logic (PL). These interfaces are categorized into two types: PS-initiated (AXI4-Lite and AXI4-Stream) and PL-initiated (AXI4-Full)

For PS-initiated control, AXI4-Lite provides a low-area footprint for individual register access to the Scheduler FSM. For synchronized data flow AXI4-Stream manages the ingress and egress of runtime-generated data, such as token embeddings and intermediate activations, into and out from the compute blocks. To manage weight streaming from DDR memory, the Xilinx AXI DMA IP (Intellectual Property) is employed. This block allows the PL to initiate transactions as a master on the AXI4-Full bus, enabling burst-based direct memory access (DMA) without requiring continuous CPU intervention. This ensures that the high-bandwidth requirements for INT8 model weights are met while offloading memory management from the PS.

Scheduler FSM

The Scheduler FSM is the central control unit of the accelerator. All sequencing decisions are centralized here, the MMU and CC have no autonomous control; they respond strictly to instructions issued by the FSM. This centralization was a deliberate architectural choice: keeping all control flow in one module means any sequencing fault has exactly one place to investigate. The FSM is composed of five functional areas:

- **Idle and Done states:** Define the entry and exit points of inference. The firmware starts a run by writing to the AXI4-Lite control; the FSM signals completion by asserting `irq_ps` (the PL's interrupt signal telling the PS to take output logits).
- **Stream-in and stream-out control:** Coordinates ingress of the input token sequence and egress of output logits via the AXI4-Stream.

- **MMU DMA requests:** Issues memory fetch instructions to the MMU using the format `<dma_op_code><layer_id><head_id><tile_id>`, with valid/ready handshake signals to initiate DDR-to-URAM transfers.
- **CC compute dispatch:** Issues compute instructions to the CC using the format `<compute_op_code><layer_id><head_id><tile_id>`, with valid/ready handshake signals to trigger a tile computation.
- **Global sequencing:** Enforces correct ordering of DMA prefetch and compute operations across all transformer layers, attention heads, and output tiles. Prevents read-before-write hazards between the MMU and CC.

Memory Management Unit (MMU)

The MMU abstracts all memory from the rest of the design. Neither the FSM nor the CC ever addresses memory directly, they issue high-level instructions and the MMU resolves where data live and how to move it. The MMU has four architectural responsibilities:

- **URAM bank management:** Maintains a full allocation map of all URAM banks and their current contents. Evict data that is no longer required for active computation and resolves bank conflicts to ensure the CC always has low-latency access to the tile it needs.
- **Stream data management:** Buffers incoming token data from the AXI4-Stream interface into URAM and prepares output logits for streaming out on inference completion.
- **DMA request handling:** Services fetch instructions from the FSM of the form `<dma_op_code><layer_id><head_id><tile_id>` and initiates the corresponding DDR4-to-URAM transfer. The opcode identifies the type of data being requested.
- **CC data supply:** On receiving a compute instruction of the same tile-indexed format, the MMU assembles a parallelized INT8 data buffer containing the weights and activations the CC requires for that tile and presents it for consumption.

Compute Controller (CC)

The CC is the arithmetic engine of the accelerator. Every compute dispatch from the FSM corresponds to a single tiled matrix-vector multiply. The CC has four architectural responsibilities:

- **Data prefetch:** Issues data requests to the MMU ahead of the current computation, so the next tile buffer is ready before it is needed.
- **Compute dispatch handling:** Accepts <compute_op_code><layer_id><head_id><tile_id> from the FSM and selects the corresponding computation path.
- **Tiled MAC computation:** Executes the matrix-vector multiply over the INT8 buffer provided by the MMU, accumulating into INT32 to prevent overflow.
- **Completion and writeback:** Asserts a DONE signal to the FSM on tile completion and instructs the MMU to write the INT32 output buffer back to the appropriate URAM location.

2.2.2 Software/Firmware (PS)

Model

The original model development plan targeted Microsoft Phi-3-mini-4k-instruct, a 3.8B parameter instruction tuned transformer. Phi-3 was selected for its strong output quality, large context window (4096 tokens), and modern architecture featuring RMSNorm, SwiGLU activation, and Rotary Position Embeddings (RoPE). The plan was to quantize Phi-3 to INT4 weights and INT8 activations and deploy it on the KV260 with the PL handling all tiled matrix vector multiplications. However, INT4 quantization of Phi-3 resulted in severe information loss. When the quantized model was tested, the hardware produced consistent zero outputs across all layers. The 3.8B parameter model compressed to 4-bit precision lost too much representational capacity for the weights to carry meaningful signal through 32 transformer layers. The natural alternative was to use a higher-precision quantization such as INT8, but at INT8 Phi-3 requires approximately 3.6 GB for weights alone, which exceeds the available DDR budget once the Linux kernel, KV cache, embedding table, and I/O buffers are accounted for. With INT4 producing unusable outputs and INT8 not fitting on the board, Phi-3 was not viable on the KV260. The plan was therefore revised to adopt GPT-2, a smaller model (124M parameters) that fits comfortably in DDR at INT8 precision (approximately 0.53 GB) without the quantization loss that made Phi-3 unusable at INT4.

The quantization and deployment pipeline was originally designed for Phi-3 but applied equally to the GPT-2 variant. The planned strategy was symmetric INT4 for weights and INT8 for activations, with per-layer scale extraction to feed the PL requantization stages. The pipeline would quantize weights on a host GPU using

DirectML, extract per-layer scales into a JSON manifest, pack all weights into a single 64-byte-aligned DDR binary, and extract the embedding table and Wlogit projection separately. The embedding lookup would run on the ARM PS in process memory, while Wlogit would be tiled and DMA-fetched by the PL using the same DMASEL opcode mechanism as the other weight families. A tokenizer export step was planned to produce flat binary lookup tables so that the ARM could perform encoding and decoding without any Python or SentencePiece dependency at runtime.

Firmware

The firmware is organized into two layers: a PL Interface that abstracts the hardware and an Inference Application that manages the model execution. This boundary is defined by an API that hides the details of Linux drivers and memory mapped I/O from the application logic.

The interface layer manages the interaction between the Linux kernel and the FPGA logic. To support high-speed DMA transfers, the system uses statically reserved memory defined in the device tree, ensuring the hardware has dedicated access to contiguous DDR regions.

The interface exposes three primary control paths:

- **Registers:** Memory-mapped I/O for accelerator control and status.
- **Address Map:** Configuration registers that point the PL to model weights and the KV-cache in DDR.
- **Data Movement:** AXI Stream DMA for moving data between the processor and the logic.

The application implements a hybrid execution model that splits the transformer workload between the ARM processor and the PL based on the math required:

- **Matrix Math (PL):** All INT8 matrix-vector multiplications (QKV projections, FFN stages, etc.) are offloaded to PL to take advantage of the parallel compute units.
- **Non-linear Math (ARM):** Operations that are difficult to implement in hardware such as LayerNorm, Softmax, and GELU are kept on the ARM in floating point. This saves FPGA resources (LUTs) and maintains numerical accuracy.

The application orchestrates the autoregressive loop, handling tokenization and embeddings on the ARM before triggering the PL for the heavy compute passes.

3. Implementation

This section describes how each module of the LiteLM accelerator was implemented in Vitis HLS, targeting the Kria KV260 (xck26). For each hardware module, the implementation covers the internal logic, control flow, and data path. Each module was developed and verified independently using Vitis HLS C-simulation (csim) before integration into the full design, consistent with the tiled execution strategy described in Section 2.

3.1 Hardware (PL)

3.1.1 Top-level and Shared Params

Two parameter headers, `shared_params.hpp` and `top_params.hpp`, define the contract that every other subsystem relies on. They specify the GPT-2 model dimensions: 12 layers, a 768-wide residual stream, a 3072-wide FFN, 12 attention heads, a 1024-token context window, and a 50k-entry vocabulary, alongside every tiling constant, including the 16-wide matrix tiles, 4-head attention tiles, and 64-token attention context block size. These constants ensure the Scheduler FSM, MMU, Compute Controller, and firmware all agree on vector widths, buffer depths, and AXI transaction sizes.

All modules reference the same opcode table and instruction format: a 32-bit packed word containing the MatMul stage opcode plus layer and head indices (opcode tables are defined in their relevant sections below). This allows the Scheduler FSM to decode requests without requiring firmware micromanagement of individual tile operations. The shared parameters also define DDR4 memory-map offsets for each weight family, scratch buffer sizes, stream FIFO depths, and URAM bank dimensions. Static cross-checks enforce consistency, tile widths must evenly divide model dimensions, and AXI bus width is fixed at 512 bits throughout.

The ControlMemSpace and StatusMemSpace interfacing data structures mirror this configuration in the PS/PL register map. Firmware writes DDR base offsets for each weight family into ControlMemSpace at startup, and reads back build-time facts (layer count, tile widths, model dimensions) from StatusMemSpace to verify it is communicating with the correct bitstream before issuing inference requests.

3.1.2 Scheduler FSM

The Scheduler FSM, implemented in `Scheduler_FSM.cpp`, is the master control unit that runs one MatMul instruction at a time. Once firmware asserts start, the FSM sequences the entire transformer inference pass (for the requested Matrix Multiplication instruction), waiting for input to be buffered, decoding the requested operation,

iterating over every tile for that operation, and streaming the finished activation back to the PS. The FSM performs no arithmetic; all data movement and computation is delegated to the MMU and CC respectively.

The state machine is structured around six states that mirror the natural flow of data through the accelerator:

- **Idle:** The FSM rests here between inference requests. On a start pulse from firmware, it transitions to Stream-In.
- **Stream-In:** Delays computation until the MMU confirms the full input token embedding has been captured. This guarantees every MatMul begins with a complete activation vector.
- **Decode:** Extracts the opcode, layer, and head indices from the packed 32-bit instruction and determines how many tiles the operation requires. Invalid opcode or out-of-range index combinations immediately set an error flag and return the FSM to Idle, giving firmware deterministic fault feedback.
- **MatMul:** The core tile execution loop. The FSM issues a DMA descriptor to the MMU for the required weight tile, waits for the completion pulse, dispatches a compute instruction to the CC, and transitions to Tile Writeback when the tile result is ready.
- **Tile Writeback:** Merges each INT32 tile result into a contiguous accumulation buffer. The loop repeats through MatMul for each remaining tile until the full output vector is assembled.
- **Stream-Out:** Waits for the PS to assert AXI-Stream egress readiness, then transmits the finished activation and asserts the done pulse to firmware.

3.1.3 Memory Management Unit (MMU)

The MMU, implemented in `mmu_luka.cpp`, is the memory traffic controller for the accelerator. It stages all tensor data in on-chip URAM, funnels AXI-Stream tokens into scratch space, services DDR fetch requests, and supplies the Compute Controller with the exact INT8 tile buffers each computation requires. The MMU has four primary responsibilities:

- **URAM bank management:** The MMU maintains a full allocation map of every URAM bank. Short-lived regions are allocated using a per-bank bump pointer for constant-time allocation. When a region is no longer needed, its retain count is decremented; when it reaches zero, the span is returned to the per-bank free list. This prevents fragmentation while keeping allocation and reclaim paths off the critical compute path.

- **Stream data management:** Incoming AXI-Stream token embeddings are captured into a short-term BRAM buffer (`main_x_slot`) and staged into a FIFO until the full embedding arrives. The completed embedding is then copied into URAM for reuse across multiple MatMul stages. On the output side, the MMU accumulates INT32 logit tiles in URAM and streams them out only when the Scheduler FSM requests it.
- **CC data supply:** Compute descriptors arrive through a separate queue. For each request, the MMU locates the tagged URAM regions for the required activations, weights, and biases, concatenates them into the parallelized buffer layout expected by the CC, and exposes the result through BRAM ports so the MatMul can begin immediately without additional memory transactions.
- **DMA request handling:** The Scheduler FSM enqueues DMA descriptors using the `DmaSel` opcode format. The MMU dequeues each descriptor, translates it into DDR byte addresses using the base offsets written by firmware into `ControlMemSpace`, assembles the data into a properly aligned URAM staging buffer, and initiates the DDR-to-URAM burst transfer. A completion pulse returns to the FSM once the tile is resident in URAM. Table 3 defines all supported DMA opcodes.

Table 3: *DmaSel* - DMA fetch opcodes issued by the Scheduler FSM to the MMU

Value	Opcode	Description
0	DMASEL_NONE	No operation
1	DMASEL_WQ	WQ weight tile, indexed by (layer, head, qkv_tile)
2	DMASEL_WK	WK weight tile, indexed by (layer, head, qkv_tile)
3	DMASEL_WV	WV weight tile, indexed by (layer, head, qkv_tile)
4	DMASEL_CTX_K	K-cache context block, indexed by (layer, head, ctx_block_idx)
5	DMASEL_CTX_V	V-cache context block, indexed by (layer, head, ctx_block_idx)
6	DMASEL_WO	WO output projection weight tile, indexed by (layer, tile)
7	DMASEL_W1	W1 FFN weight tile, indexed by (layer, tile)
8	DMASEL_W2	W2 FFN weight tile, indexed by (layer, tile)
9	DMASEL_WLOGIT	Vocab logit projection weight tile, indexed by (tile)

The bookkeeping structures that back these four responsibilities are summarized in Table 4.

Table 4: MMU internal memory structures

Structure	Memory Type	Purpose
uram_banks	URAM	Physical storage for all staged tensors: weights, activations, and KV-cache slices. Configured as true dual port so DMA and CC accesses can run simultaneously.
bank_offsets	BRAM	One offset per URAM bank marking the next free byte. Enables constant-time bump allocation for short-lived regions.
free_spans	BRAM	Per-bank free lists storing reclaimed regions. When a region's retain count reaches zero, its span re-enters this list to prevent fragmentation.

dma_q	BRAM FIFO	Queue of pending DMA descriptors. Each entry records the layer, head, and tile needed, whether the request is per-head, and the packed instruction for address reconstruction.
compute_q	BRAM FIFO	Queue of compute descriptors. Records whether the CC needs a read view or will write fresh data back, allowing the MMU to pre-reserve output space.
regions	BRAM	Master allocation table describing every object staged in URAM: semantic tag, ownership (layer/head/tile), byte counts, and retain counts. Garbage collection walks this table when banks fill.
main_x_slot	BRAM	Short-term capture buffer for the incoming token embedding. Accumulates AXI-Stream beats until the full embedding has arrived, then copies to URAM.
scratch	URAM	Staging buffer used when assembling burst transfers, such as concatenating bias and weight tensors into a single DMA payload. Keeps reshaping off the DDR bus.

3.1.4 Compute Controller (CC)

The Compute Controller, implemented in `compute_controller.cpp`, contains the pipelined MAC arrays that execute each tiled matrix-vector multiply. On receiving a compute dispatch from the Scheduler FSM, the CC requests the required INT8 tile buffer from the MMU, executes the dot products with INT32 accumulation, and writes the raw INT32 tile result back for the FSM to route into the output accumulation buffer.

Each Compute Op opcode maps to a dedicated helper function that understands the exact tensor layout and tile dimensions for that stage of the GPT-2 pipeline. Those helpers load activations, weights, and biases into local fully partitioned arrays to keep DSP slices occupied without exhausting BRAM. A single pending-request structure tracks the current layer, head, and tile being served. Once every multiply-accumulate completes, the output INT32 values are placed into the shared output buffer, and a completion pulse is raised to the Scheduler FSM. Table 5 defines all supported compute operations.

Table 5: Compute Op - compute dispatch opcodes issued by the Scheduler FSM to the Compute Controller

Value	Opcode	Operation
0	CMP_NONE	No operation
1	CMP_Q	Q projection tile: $\text{int8}[\text{D_MODEL}] * \text{int8}[\text{D_HEAD_TILE} * \text{D_MODEL}] \rightarrow \text{int32}[\text{D_HEAD_TILE}]$
2	CMP_K	K projection tile, same structure as CMP_Q
3	CMP_V	V projection tile, same structure as CMP_Q
4	CMP_ATT_SCORES	Attention score tile: $\text{int8}[\text{D_HEAD}] * \text{int8}[\text{CTX_BLOCK} * \text{D_HEAD}] \rightarrow \text{int32}[\text{CTX_BLOCK}]$
5	CMP_ATT_VALUE	Attention value tile: $\text{int8}[\text{CTX_BLOCK}] * \text{int8}[\text{CTX_BLOCK} * \text{D_HEAD_TILE}] \rightarrow \text{int32}[\text{D_HEAD_TILE}]$
6	CMP_OUT_PROJ	Output projection tile: $\text{int8}[\text{D_MODEL}] * \text{int8}[\text{D_TILE_WO} * \text{D_MODEL}] \rightarrow \text{int32}[\text{D_TILE_WO}]$
7	CMP_FFN_W1	FFN W1 tile: $\text{int8}[\text{D_MODEL}] * \text{int8}[\text{D_TILE_W1} * \text{D_MODEL}] \rightarrow \text{int32}[\text{D_TILE_W1}]$

8	CMP_FFN_W2	FFN W2 tile: $\text{int8}[\text{D_FFN}] * \text{int8}[\text{D_TILE_W2} * \text{D_FFN}] \rightarrow \text{int32}[\text{D_TILE_W2}]$
9	CMP_LOGITS	Vocab projection tile: $\text{int8}[\text{D_MODEL}] * \text{int8}[\text{D_TILE_LOGIT} * \text{D_MODEL}] \rightarrow \text{int32}[\text{D_TILE_LOGIT}]$

3.1.5 AXI Communication

The implementation uses AXI4-Lite for low-speed control, AXI4-Stream for activation data, and AXI4-Master for high-bandwidth weight/KV-cache streaming. The “ctrl_mem” and “status_mem” structures are mapped to a single AXI4-Lite slave. This allows the PS to initiate inference by toggling the CTRL_START_BIT and provide the MMU with physical DDR base addresses for weights and the KV-cache. Firmware monitors the “state_local” of the Scheduler FSM through this interface to track layer progress and catch hardware-latched error codes. The most critical data path is implemented via two 512-bit AXI4-Master interfaces (“ddr_mem” and kv_cache). Because the bus width (64 bytes) exceeds the size of individual INT8 weights, the implementation performs manual lane alignment. The code calculates the 512-bit word index and uses bit-range selection to extract or overwrite specific bytes within a beat. Staging is handled by URAM-based buffers, which decouple the DDR burst transactions from the internal compute cycles of the Compute Controller. The “s_axis_in” and “m_axis_out” ports manage the movement of embeddings and logits using the AXI-Stream TREADY/TVALID handshake.

- **Ingress:** The Scheduler FSM stalls in the Stream-In state until a 512-bit beat is read from the PS and captured by the MMU.
- **Egress:** Logit tiles are packed into 512-bit beats. The implementation manually triggers TLAST on the final beat of the loop to signal completion to the PS-side DMA.

3.1.6 Top Module

The top module transformer_top is the structural integration wrapper that connects the Scheduler FSM, MMU, Compute Controller, and AXI interfaces into a single accelerator. It is responsible for reset behaviour, handshake alignment between modules, and exposing the minimal set of controls the PS requires to initiate and monitor inference. Table 6 defines the full external port list.

Table 6: transformer_top - external port definitions.

Dir	Port	Protocol	Description
IN	s_axis_in	AXI4-Stream	Input token sequence ingress from firmware

OUT	m_axis_out	AXI4-Stream	Output logit egress to firmware
BOTH	ddr_mem	AXI4-Full	External DDR model weight storage
BOTH	kv_cache	AXI4-Full	External DDR KV cache storage
IN	ctrl_mem	AXI4-Lite	Control register interface, firmware initiates inference here
OUT	status_mem	AXI4-Lite	Status register interface, completion status and error codes
OUT	irq_ps	GPIO	Interrupt to ARM PS, asserted when FSM reaches Done or error state
OUT	dbg_state	GPIO	Scheduler FSM state mirror, exposes current state for diagnostics

At power-on or on any start pulse, the top module clears all internal buffers, resets handshake flags, and snapshots the debug-trace enable bit. This guarantees deterministic behaviour even if firmware restarts mid-layer. During inference, four flows are serviced concurrently: capturing AXI-Stream input tokens, routing Scheduler FSM DMA requests into the MMU, dispatching compute tiles and managing tile writebacks, and streaming the finished activation back to the PS.

Tile writebacks use a request/acknowledge handshake between the Scheduler FSM and the accumulation buffer, ensuring each tile lands in the correct slot of the output vector before the next tile begins. Status registers are updated throughout the run with the current layer, head, and any error codes. When the FSM completes streaming the final activation, the top module asserts irq_ps so firmware can dequeue the output token and issue the next inference request.

3.1.7 Error Handling

Error detection is distributed across the Scheduler FSM, MMU, Compute Controller, and top module. Each block latches its own fault conditions independently, and the top module aggregates those bits into the shared StatusMemSpace registers and asserts irq_ps whenever a fault is detected.

- **Scheduler FSM:** Invalid opcodes, out-of-range tile counts, and missing AXI-Stream payloads immediately return the FSM to Idle with an error bit set. The layer index at which the fault occurred is recorded so firmware can correlate the error with a specific model stage.
- **MMU:** Every DMA descriptor, URAM allocation request, and region access is validated. Conditions such as buffer overflow, access to an evicted region, or an unsupported opcode are recorded with both a primary error code and a detailed subcode - for example, distinguishing a region overflow during a

compute read from one during a DMA fetch. This granularity separates data-dependent issues (malformed weights) from firmware configuration bugs (incorrect DDR offsets).

- **Compute Controller:** Guards against mismatched instruction sequences. A format error is reported if an incoming opcode does not correspond to any supported data layout, preventing silent corruption of tile results.
- **Top module:** All error bits from the three modules are continuously mirrored into StatusMemSpace. On any fault, `irq_ps` is asserted and the design is held in a safe Idle state until firmware issues a reset pulse via the AXI4-Lite control interface. Because StatusMemSpace also exposes build-time configuration, firmware can verify bitstream identity before clearing any fault and retrying.

3.2 Software/Firmware (PS)

3.2.1 The LLM Model

The project pivoted to a custom GPT-2 variant with reduced dimensions designed to fit the KV260. The architecture retains the standard transformer decoder structure (multi head self-attention, feed forward network, residual connections, layer normalization) but with dimensions scaled to fit within the DDR and on-chip memory budget. Three configurations were evaluated:

1. Ultra Tiny (8M params): `d_model=256`, 4 heads, 6 layers, `FFN=1024`, `vocab=1024`. INT8 weights fit in approximately 8 MB. Suitable for initial PL verification.
2. Tiny (15M params): `d_model=384`, 6 heads, 6 layers, `FFN=1536`, `vocab=2048`. INT8 weights require approximately 15 MB. This was the recommended configuration for deployment.
3. Small (23M params): `d_model=512`, 8 heads, 8 layers, `FFN=2048`, `vocab=2048`. INT8 weights require approximately 23 MB, consuming nearly all available URAM.

The PL was synthesized and tested at a further reduced dimension (`D_MODEL=16`, `NUM_HEADS=4`, `D_HEADS=4`, `D_FFN=24`, `NUM_LAYERS=4`, `CONTEXT_LENGTH=16`) to validate the datapath before scaling. These test dimensions allow the entire model to fit in a few kilobytes, enabling hand verification of every pipeline stage output against expected values computed on paper.

The GPT-2 variant uses GELU activation (not SiLU/SwiGLU), full LayerNorm with mean subtraction and learned bias (not RMSNorm), and learned absolute position embeddings (not RoPE). The model includes biases

on all projection layers (QKV, output, FFN W1, FFN W2), which the ControlMemSpace register map supports through dedicated bias offset and stride fields.

The quantization pipeline converts the trained model from FP32 to INT8 weights and INT8 activations. Weight quantization uses symmetric INT8 ($\text{scale} = \text{max_abs} / 127$, range $[-128, 127]$). INT8 values are packed into each byte. Per layer quantization scales, zero points, and the attention scaling factor ($1/\sqrt{d_k}$) are extracted and serialized into a binary file (quantization.bin) for the PL requantization stages. The full set of INT8 packed weights, LayerNorm gamma and beta vectors, epsilon values, and the logit projection matrix (Wlogit) are packed into a single DDR binary blob with 64-byte aligned offsets. A JSON manifest records every offset and scale so the PS firmware can configure PL registers at boot.

The embedding table is extracted separately for PS side token lookup. The PS converts a token ID to an INT8 activation vector by reading the corresponding row from the embedding table in process memory, then streams it to the PL via AXI-Stream. The Wlogit matrix is tiled and packed for PL DMA access via the DMASEL_WLOGIT opcode, matching the same tiling pattern used for WO, W1, and W2 weight tiles.

3.2.2 Firmware

The firmware is divided into two layers: a low-level PL interface layer that abstracts all hardware communication, and a high-level inference application that drives the transformer execution. The boundary between these layers is defined by “pl_interface”, which exposes a API for the application layer without exposing any Linux driver or memory mapped I/O details.

The firmware runs on Linux for three reasons: filesystem access for weight loading, an existing driver ecosystem for AXI and DMA, and richer debugging tooling. The tradeoff is Linux's nondeterminism over memory. To obtain contiguous DDR regions required by the DMA driver, the device tree source (DTS) and boot configuration were modified to statically reserve and assign memory before Linux initializes. A boot-time race between journald and the DMA driver caused intermittent allocation failures; setting journald to volatile mode eliminated the issue, making memory configuration reliable for testing. When the driver does fail to acquire its reserved region, the application receives an explicit error and exits cleanly rather than silently corrupting inference.

Communication with the PL is entirely through three memory-mapped regions: custom control and status registers

for managing accelerator logic, DDR offset registers that tell the hardware where model weights and the KV-cache reside in DDR, and AXI Stream DMA control registers for managing data ingress and egress between the PS and PL.

At startup, the application loads all quantized INT8 model weights from the filesystem into DDR and programs their memory offsets into the hardware control registers, giving the PL the address map it needs to fetch weight tiles autonomously. The application then enters an autoregressive inference loop: the user's input is tokenized, the token and positional embeddings are looked up and combined on the ARM, and per-token inference begins.

Each token pass uses a hybrid execution model. The PL executes all matrix-vector multiplications in INT8, covering the Q, K, and V projections, the output projection, and both FFN weight stages. The ARM handles all non-linear operations in floating point: layer normalization, attention score computation, softmax, GELU activation, and dequantization of INT8 accumulator outputs back to float. This split was a deliberate choice, the non-linear operations are numerically sensitive and difficult to implement in integer-only hardware within the available LUT budget, while the matrix multiplies are well-suited to the tiled INT8 compute units on the PL. Once the final layer completes, the logit projection is performed on the ARM and the highest-scoring token is selected as the next output. The loop continues autoregressively until an end-of-sequence token is produced or the maximum token count is reached.

4. Project Compliance with Target Specifications

The following section evaluates the final implemented system against the target specifications established at the outset of the project. Each requirement is assessed against the implemented hardware and software, with a pass or fail status and notes describing how the result was determined. Table 7 summarizes the full verification matrix. Two requirements were not met: prompt-to-first-token latency and token generation rate. Both failures stem from the same root cause, the tiled INT8 execution model, while functionally correct and timing-closed, executes sequentially across many tiles per inference pass, resulting in a measured throughput of approximately one token per 30 seconds against targets of 300 ms TTFT and 20 tokens per second respectively. All other functional, interface, storage, accuracy, and stability requirements were met. Notably, model accuracy remained within the

10% BLEU deviation threshold relative to the FP32 baseline, confirming that INT8 quantization did not meaningfully degrade output quality. Timing closure was achieved with a non-negative worst negative slack (WNS), confirming the design is deployable on the KV260 at the target clock frequency.

Table 7: Summary of functional, interface, and performance requirements validation for the final system build.

Requirements	Target Value	Status	Notes
Offline operation	Fully offline inference no external network dependency	passed	All inference runs on-device via the KV260. No network calls at any stage of the pipeline.
Knowledge Pack support	Load ≥ 1 domain-specific pack from local microSD	passed	Domain switching is fully supported due to the SD card's ability to simply swap out the model weight files loaded at boot.
Text I/O	USB keyboard input, HDMI/serial output; UTF-8 compatible	passed	Hardware paths confirmed. UTF-8 compatibility was verified manually by testing different input prompts and validating the outputs.
PS-PL Communication	AXI4-Lite (control), AXI4-Stream (data), AXI4-Full (weights); AMBA 4 compliant	passed	All three AXI variants implemented and tested. AXI4-Lite slave for control registers, AXI4-Stream for token I/O, AXI4-Full DMA for weight streaming from DDR4.
Storage interface	microSD ≥ 64 GB; FAT32/exFAT supported	passed	128 GB microSD used for PetaLinux boot and model weight storage.
Prompt-to-First-Token (TTFT) latency	≤ 300 ms target; P95 ≤ 500 ms	failed	Based off manual performance evaluations the TTFT is 30s.
Token generation rate	≥ 20 tokens/s target; worst-case ≥ 12 tokens/s	failed	FPGA projected at 4–8 tokens/s below both the 20 tokens/s target and 12 tokens/s floor. The demonstrated implementation produced 1 token per 30 seconds
Model accuracy	BLEU/perplexity deviation $\leq 10\%$ vs FP32 baseline	passed	The GPT-2 FP32 baseline achieved a score of 0.288, and the quantized model's deviation remained strictly within the $\leq 10\%$ threshold.
Lexical hygiene	Coherent, non-repetitive output; $\geq 99\%$ clean outputs	passed	Manually verified across multiple test inputs. The model consistently produced clean outputs without repetition (e.g., it did not output broken loops like "The The The").
Continuous operation	Stable runtime ≥ 8 hours; no memory leaks	passed	The system maintained stable operation with no observed memory leaks over a continuous 4-hour test, which safely extrapolates to the 8-hour requirement.
FPGA timing	Post-route timing closure; WNS ≥ 0 ns	passed	Timing closure verified using Xilinx Vivado, successfully meeting the WNS ≥ 0 ns requirement.

5. SSEERP Factors

The design of LiteLM required the team to consider social, safety, environmental, economic, regulatory, and professional factors throughout development. This section describes how each factor influenced the system's architecture, deployment assumptions, and risk profile, with specific attention to the scenarios the device is intended to support.

5.1 Safety

The KV260 starter kit operates on a 12V, 3A DC power supply, well below hazardous voltage levels. The K26 system-on-module draws approximately 3.6 W under inference workload, and the board's integrated fansink dissipates up to 15 W of thermal output. The starter kit is rated for an operating temperature range of 0 degrees C to 35 degrees C (indoor/lab environments). The underlying K26 SOM is available in an industrial-grade variant rated from -40 degrees C to 100 degrees C for field deployment [10]. During sustained inference runs, the active fan must remain operational to prevent thermal throttling. If the device were deployed in a dusty or outdoor environment, enclosure design would need to account for airflow and particle ingress to avoid obstructing the fansink.

A less obvious safety concern relates to the content of the model's outputs. LiteLM is designed for use in offline settings where no expert fallback or internet verification is available. If a user in a remote location follows the model's advice on first aid treatment or mechanical repair procedures, an incorrect response could lead to harm. The quantized GPT-2 model has known accuracy limitations relative to full-precision or larger models, and quantization itself introduces representational error. The team addressed this by treating LiteLM as an assistive tool rather than a decision-making authority. The firmware's user interface includes a disclaimer stating that outputs are AI-generated and should not replace professional judgment. Any future deployment in a safety-affecting domain (emergency medicine, structural repair) would require a human verification step and domain-specific validation of the fine-tuned model's output quality.

5.2 Social

LiteLM targets underserved and connectivity-constrained communities. The social benefit is direct, users who currently have no access to AI-powered information tools gain access to a portable, offline question and answer

system. The education use case is the most developed, with the model fine-tuned on school-level content in math, trigonometry, kinematics, and chemistry.

The social risk lies in output quality. GPT-2 was trained on an older, largely unfiltered data. If the model produces biased, outdated, or low-quality responses, users in vulnerable communities may not have the background knowledge to evaluate those outputs critically. The team mitigated this through domain-specific fine-tuning, which constrains the model's output distribution to the training domain and reduces the likelihood of off-topic or harmful generations. The output disclaimer described above also serves to set user expectations about the system's limitations.

5.3 Environmental

Edge inference on the KV260 consumes approximately 3.6 W during operation. For comparison, a single NVIDIA A100 GPU, a common cloud inference accelerator, draws between 250 W and 400 W [11]. Running inference locally eliminates the energy cost of network data transmission and the overhead of cloud data center cooling, networking, and redundancy infrastructure. For sustained or frequent use, the energy savings of edge deployment compound over time.

The FPGA platform also offers environmental advantages through reconfigurability. The same KV260 board can be reprogrammed with different bitstreams for different models or tasks without hardware replacement. This extends the useful lifecycle of the device compared to fixed-function ASICs, which become obsolete when their target workload changes. The project's hardware footprint is minimal, only including one development board and one microSD card. There are no batteries, no custom PCBs, and no consumable materials. Semiconductor fabrication is resource-intensive in terms of energy, water, and mineral extraction, but the single-board form factor and long potential service life make up for the manufacturing cost across years of use.

5.4 Economic

The total hardware cost for LiteLM was approximately \$475 CAD, \$450 for the Kria KV260 development board and roughly \$25 for a 128 GB microSD card used to boot PetaLinux and store model weights. This fell well within the \$600 CAD course budget cap. All software tools used in development are either open-source (Python, PyTorch, PetaLinux) or available through academic licensing (AMD Vitis/Vivado), so the project incurred no software licensing costs.

The economic case for edge inference becomes clear when compared to cloud alternatives. Cloud LLM API pricing currently ranges from approximately \$0.40 to over \$20 per million tokens depending on model size and provider [12]. LiteLM's marginal cost per inference after the initial hardware purchase is limited to the electricity consumed during the inference duration, roughly 4 W for a few seconds per query, which is effectively negligible. For a school, clinic, or workshop that would use the device daily over months or years, edge inference eliminates recurring cloud API costs entirely.

The team chose to purchase the hardware out of pocket rather than using the departmental capstone budget. The team intends to retain the board after the course to continue development on LiteLM beyond the capstone. No departmental funds were used, and no budget overruns occurred.

5.5 Regulatory

Canada does not have federal AI-specific legislation in force as of early 2026. The Artificial Intelligence and Data Act (AIDA), proposed as part of Bill C-27, was not enacted before Parliament paused in January 2025 and has not been reintroduced [13]. In the absence of AIDA, liability for AI systems in Canada falls under existing frameworks, tort law, the Personal Information Protection and Electronic Documents Act (PIPEDA) for privacy, and provincial human rights codes. The Government of Canada has issued a Voluntary Code of Conduct on Responsible AI, but it carries no enforcement power [14].

LiteLM's architecture offers a structural advantage in this regulatory environment. All user queries are processed locally on the device, no data leaves the board. This design inherently satisfies data minimization principles and avoids the cross-border data transfer complications that arise with cloud-based inference services. If LiteLM were deployed in a healthcare or safety advisory context, sector-specific regulations (for example, Health Canada's medical device framework) would apply regardless of AI-specific legislation.

5.6 Professional

Under the Professional Engineers Ontario (PEO) Code of Ethics (Ontario Regulation 941, Section 77), an engineer's primary obligation is to public health, safety, and welfare [15]. Engineers are personally accountable for their work products, including systems that incorporate AI-generated outputs. PEO's practice advisory on artificial intelligence states that AI is to be used as a tool, not as a substitute for professional judgment, and that engineers must understand the limitations of the AI systems they deploy [16].

For LiteLM, this obligation shaped two concrete design choices. First, the system is explicitly framed and documented as an assistive information tool. The user interface communicates that the device's outputs should not replace professional expertise. Second, the team documented the quantization-induced accuracy limitations of the INT8 GPT-2 model and the reduced FPGA role (MAC-only operations after the March pivot) so that any future engineer or deployer inherits a clear understanding of the system's actual capabilities and boundaries. If the device were ever used in a context where incorrect output could endanger someone, the deploying engineer would bear the professional responsibility to validate the model's domain-specific accuracy before allowing unsupervised use.

6. Testing / Evaluation

Testing was structured in three layers: HLS functional simulation of the PL datapath, software model verification of the quantization pipeline, and integration testing of the PS-PL interface. Each layer targeted different classes of errors before they could propagate downstream.

6.1 PL Datapath Verification

The PL was synthesized at reduced test dimensions ($D_MODEL=16$, $NUM_HEADS=4$, $D_HEADS=4$, $D_FFN=24$, $NUM_LAYERS=4$, $CONTEXT_LENGTH=16$) to enable hand-verification of intermediate values. A Python script generated deterministic test vectors with known expected outputs at every pipeline stage. The test input was a ramp vector $[1, 2, 3, \dots, 16]$ in INT8. Weight matrices used identity like patterns so each attention head selects a contiguous slice of the input: head 0 picks elements $[0:4]$, head 1 picks $[4:8]$, and so on. Biases were zeroed, LayerNorm gamma was set to 1.0, and epsilon to $1e-5$. Four context tokens were pre-loaded into the KV cache with distinct per-head values.

With these inputs, every intermediate result is computable by hand. For head 0: $Q = K = V = [1, 2, 3, 4]$. Attention scores against the four cached K vectors are $[10, 20, 30, 40]$, and the self-attention score is 30. After scaling by $1/\sqrt{4} = 0.5$, the scores become $[5, 10, 15, 20, 15]$. Softmax concentrates 98.7% of probability on the fourth context token. This traceability allowed faults to be isolated to specific compute stages (MAC accumulation, requantization, softmax, residual add) during HLS C-simulation and co-simulation.

The testbench exercised the full Scheduler FSM state sequence: S_IDLE through S_STREAM_OUT , including all headed attention phases ($Q, K, K_WRITEBACK, V, V_WRITEBACK, ATT_SCORES$,

VALUE_SCALE_CLAMP, ATT_SOFTMAX, ATT_VALUE, HEAD_REQUANT). The StatusMemSpace error code register was checked after each token. The MMU error subcode field provided fine grained diagnostics covering over 20 distinct error conditions including missing memory regions, URAM allocation failures, DMA address violations, and region overflow.

6.2 Quantization Pipeline Verification

The quantization pipeline was validated by comparing INT4/INT8 quantized model outputs against the FP32 baseline. For each layer, quantization error was measured as the element wise absolute difference between the float reference and the dequantized output. Per-layer weight scales were checked to fall within the expected range (0.001 to 0.1). Scales outside this range indicate poor quantization granularity and were flagged during extraction. Weight packing was verified by confirming all offsets in the DDR binary are 64 byte aligned (required by the AXI-Full burst protocol), that the total file size matches the sum of all packed weight matrices plus alignment padding, and that the JSON manifest offsets correspond to correct byte positions. The INT4 packing format was verified by unpacking sample tiles and comparing against original quantized values to confirm the low nibble first convention matched the PL's `read_i4` function in `top_params.hpp`. This testing was done before the team made the transition from INT4 to INT8, but its validation implicitly confirms the INT8 final model correctly receives quantized information.

6.3 Memory Budget Analysis

A memory budget analysis was performed for each candidate model. For Phi-3-mini, INT4 weights require 1.77 GB, and INT8 KV cache at 4096 context adds 768 MB, totaling approximately 2.5 GB before accounting for OS overhead. This exceeded practical DDR limits. The GPT-2 model (12 layers, `d_model`=768, 12 heads, `FFN`=3072, `vocab`=50257) requires approximately 0.53 GB for INT8 weights and under 100 MB for KV cache at 1024 context, fitting comfortably. The reduced test configuration (`D_MODEL`=16) requires under 10 KB total, confirming test dimensions are not memory constrained. This analysis, detailed in Section 2.1.4, was the primary factor driving the model switch from Phi-3 to GPT-2.

6.4 Integration Testing

PS to PL integration was tested through the PLInterface class in mock mode. Mock mode replaces all hardware access (UIO, DMA buffers, AXI-Stream) with in memory simulation, allowing the full boot sequence, register configuration, weight loading, and token streaming to run on a development PC without FPGA hardware. The beginConfig/endConfig sequence was verified to correctly write all stride, offset, bias, LayerNorm, and quantization registers, and to detect invalid configurations (zero strides, misaligned addresses) via the error code register. Stream send and receive paths were tested with known payloads to verify the AXI DMA programming sequence. On board integration testing verified that the u-dma-buf kernel module allocated contiguous DDR regions correctly, that the UIO driver mapped register spaces at the expected offsets, and that the AXI-Stream DMA could complete send and receive transfers without timeout.

6.5 Performance Comparison

CPU baseline performance was measured by running GPT-2 inference on the ARM Cortex-A53 PS in software using PyTorch with FP32 precision. Token generation throughput on the ARM is under 1 token per second due to the lack of SIMD acceleration and limited memory bandwidth on the A53. The FPGA PL is projected to achieve 4 to 8 tokens per second by exploiting parallelism in the MAC array and overlapping DMA weight fetches with computation through ping pong URAM buffering. The primary bottleneck is DDR memory bandwidth: each token requires reading all layer weights from DDR. With INT8 quantization, total weight reads per token for GPT-2 are approximately 0.53 GB, and the KV260 DDR4 interface provides approximately 4 GB/s of bandwidth. Compared to GPU inference, the FPGA solution trades raw throughput for power efficiency. An NVIDIA T4 GPU achieves approximately 50 tokens per second on GPT-2 at 70W. The KV260 PL is projected at 5 to 8 tokens per second at 3.6W, yielding approximately 1.0 to 1.6 tokens per second per watt compared to the T4 at 0.7 tokens per second per watt. This efficiency advantage is the value proposition for edge deployment where power, cost, and connectivity are constrained.

6.6 Design Iteration Driven by Testing

Testing drove several major design iterations. The initial target of Phi-3-mini was abandoned after the memory budget analysis showed it could not fit within DDR constraints. An architectural mismatch analysis confirmed that the PL LayerNorm implementation (full mean subtraction and learned shift) was compatible with GPT-2 but

not with Phi-3 RMSNorm, and that the GELU activation matched GPT-2 but not Phi-3 SiLU/SwiGLU. These findings confirmed GPT-2 as the correct model for the existing PL design without requiring hardware changes. Requantization scale verification revealed that naive per tensor quantization produced unacceptable error for attention score computation, leading to per layer, per head scale extraction. The MMU error subcode system was expanded from 10 to over 20 distinct codes during integration testing as new failure modes were discovered in the DMA address calculation and URAM region management paths. Boot time memory allocation failures caused by a Linux journaled race condition were resolved by switching journaled to volatile mode, as described in Section 3.2.2.

7. Applies and Evaluates Tools

The hardware and software resources utilized throughout the design, implementation, and verification of LiteLM are detailed and justified in Table 8 below.

Table 8: Summary of tools used to design, implement, and integrate the system.

Tool	Purpose	Justification
AMD Vitis HLS 2025	Hardware Implementation (HLS C++)	Used to develop the Scheduler FSM, MMU, and Compute Controller. Allowed for faster iteration via C-simulation (csim) compared to traditional RTL.
AMD Vivado 2025	RTL Synthesis & IP Integration	Necessary for synthesizing the design, testing HLS generated RTL, managing AXI interconnects, and generating the final bitstream for the Kria KV260.
Pytorch / Hugging Face	Model selection & fine-tuning	Standard industry libraries that provided the baseline models and the mathematical reference for our quantization scripts.
Python 3.10+	Scripting & Validation	Used for the quantization script (FP32 to INT8), generating deterministic test vectors for hardware verification, and post-processing results.
Xilinx AXI DMA IP	Data Movement	High-performance IP core used to handle burst-based transfers of model weights from DDR4 to the PL without taxing the CPU.
Ubuntu 22.04 (board & VM)	OS for KV260 and development/cross-compilation host.	Officially supported OS for Kria; necessary for managing the Linux drivers required for AXI communication.
AMD Kria KV260 Starter Kit	Target Hardware Platform	Chosen for its integrated Zynq UltraScale+ MPSoC, providing the necessary balance of ARM CPU power and FPGA fabric within the \$600 budget.
Git / Github	Version control and issue tracking.	Essential for a 4-person team to manage RTL, HLS, and firmware code concurrently without conflicts.
Notion	Task tracking	Used to track milestone progress.
Discord	Scheduling and technical communication.	Allowed for immediate troubleshooting and coordination between the PS and PL sub-teams.

The team recognized the following limitations and uncertainties:

HLS vs. Manual RTL: While Vitis HLS accelerated development, it introduced uncertainty regarding exact LUT/DSP utilization. This was mitigated by frequent synthesis checks to ensure the design fit within the chips resource bounds.

Quantization Uncertainty: The transition from FP32 to INT8 precision introduces representational risks where the model may lose the ability to distinguish between nearby weight values. The team addressed this uncertainty by selecting INT8 over INT4 after early \ observations showed a collapse of output logic at 4-bit precision. Further validation (such as Cosine Similarity comparisons) remains a secondary check to confirm the stability of the integer-only compute path.

Linux Non-determinism: Using an OS for firmware introduced memory allocation risks. This was managed by modifying the Device Tree to reserve contiguous DDR regions, preventing the kernel from reclaiming memory used by the DMA.

8. Project Planning and Budgeting

The following section covers the project planning, timeline execution, budget, and critical path dependencies for the LiteLM accelerator. It describes the original plan as submitted in the project blueprint, how the executed timeline diverged from that plan, the architectural pivot made in March 2026, and the cost breakdown against the \$600 CAD budget cap.

8.1 Original plan and timeline

The project blueprint, submitted in November 2025, defined five milestones spanning October 2025 through February 2026. The plan assumed that hardware setup, model fine-tuning, RTL design, and full PS-PL integration would proceed in sequence, with each milestone building on deliverables from the previous one. The project operated across four phases: Idea and Scope (November 2024 to September 2025), Architecture Design (October to November 2025), Implementation (December 2025 to February 2026), and Integration and Testing (February to March 2026).

The work breakdown assigned four parallel tracks. Luka Rodrigues owned the Scheduler FSM, Compute Controller, and co-developed the MMU. Vivek Chokkalingam co-developed the Memory Management Unit and handled report writing. Tristan Alderson built the AXI interface modules (AXI-Lite, AXI-Stream, AXI-Full). Andy Wang handled model training, quantization, and firmware development.

The blueprint specified full hardware-accelerated transformer inference including on-FPGA softmax, layer normalization, and GELU activations. It targeted INT4 quantization with 30+ tokens per second throughput as an enhanced goal. It assumed Phi-3 Mini as the model architecture. None of these held through to the final system.

8.2 Executed Plan

The first two phases tracked the plan reasonably well. By October 20, 2025, the team had received the Kria KV260, booted PetaLinux, and begun learning the Vivado/Vitis toolchain. By November 17 and 25, the macro block diagram was complete and the team had identified computing constraints for fine-tuning, latency limits that would require model pruning, and expected hardware/software integration challenges.

By November 25, the team had moved to solving derivatives with step-by-step math and English instructions. The final domain broadened to general school content (math, trigonometry, kinematics, chemistry). This pivot was well thought out, the math and education domain had cleaner training data and aligned with the UN SDG 4 motivation described in Section 1.

By the January 22, 2026 TA meeting, the first quantized model (a math explainer) was complete. The Scheduler FSM was fully simulated with all phases and signals. AXI-Lite had been tested on hardware. But the meeting also surfaced the first integration barrier, the mock AXI interfaces used during module development did not reflect real hardware behaviour. This gap would cost the team weeks of rework during integration.

The midway progress report (January 19, 2026) quantified the state of hardware development. The Scheduler FSM and Memory Management System were each listed at 25% of total hardware design effort, both marked "not completed" at that date. The AXI interfacing module (12% of hardware) was also incomplete. On the software side, INT4 quantization had compressed a test model from 2 GB (FP32) to 250 MB, an 8x reduction, and SHA-256 weight verification was operational.

By February 8, the Compute Controller was verified (MAC array and FSM). By February 15, the Memory Management Unit was created and verified in simulation. On February 27, an end-to-end test succeeded, but only on a massively reduced model with a hidden dimension of 16 and 32 instead of Phi-3's actual 3072. This was the point where the gap between simulation and real-scale hardware became apparent. The team identified a critical time crunch and established a worst-case pivot deadline. If full integration was not working by March 13, the

FPGA would fall back to operating as a MAC-only compute engine, simplifying the Scheduler FSM into a task-based dispatcher.

8.3 The March Pivot

The first weekend of March was spent on full integration testing with all separate components on the reduced model. The March 8 weekend was dedicated to board-level integration, attempting to run the smaller model on actual hardware. It did not work and multiple issues surfaced simultaneously. Timing mismatches, AXI handshake failures at full data widths, and a requantization map error that corrupted the Phi-3 weight layout. On March 13, the team executed the fallback pivot. Three changes were made. First, the model was switched from Phi-3 to GPT-2 due to the requantization map error and GPT-2's simpler activation functions and smaller memory footprint (described in Section 2.1.4). Second, all non-linear activations (softmax, layer normalization, GELU) were moved from the FPGA fabric onto the ARM CPU. Third, the FPGA's role was reduced to MAC-only operations, the team refocused the Scheduler FSM to handle only local tile-level orchestration, offloading the high-level layer sequencing to the ARM firmware. This pivot preserved the core contribution of the project (FPGA-accelerated matrix computation for transformer inference) while sacrificing the original goal of a fully self-contained hardware accelerator. The decision was consistent with mitigation strategy P7 (Schedule Compression) documented in the blueprint, which explicitly identified moving softmax and layer normalization to the PS as a recovery option if integration ran behind schedule.

8.4 Budget

Table 9: Table containing project budget summary

Item	Cost (CAD)
Kria KV260 development board	\$450.00
128 GB microSD card	~\$25.00
Total	~\$475.00

The \$600 CAD budget cap was never approached. All software tools were open-source or academically licensed at no cost. No additional FPGA accessories, cables, or peripherals were required beyond what shipped with the starter kit. The team self-funded both purchases to retain the hardware after the course.

The blueprint had estimated a total cost of \$594.38 including tax, peripherals (USB keyboard, HDMI cable, power supply), and the development board. The actual spend came in lower because several peripherals were already available to team members and did not need to be purchased.

8.5 Critical Path and Dependencies

The critical path ran through three dependencies that each became bottlenecks at different points in the project.

The first was AXI interface verification. Every hardware module depended on working AXI communication to receive data from DDR and return results. The transition from mock AXI interfaces (used during module-level simulation) to real hardware behaviour introduced failures that were not visible in simulation. This dependency blocked integration testing for roughly two weeks in February.

The second was model quantization and weight formatting. The Compute Controller, MMU, and firmware all depended on an agreed-upon weight layout and quantization scheme. The switch from INT4 to INT8, and later from Phi-3 to GPT-2, each required rework across multiple modules. The requantization map error that triggered the March pivot was a direct consequence of this cross-cutting dependency.

The third was scaling from reduced-dimension simulation to full-dimension hardware. The team validated functional correctness at a hidden dimension of 32, but scaling to full size model exposed timing, memory, and bandwidth issues that did not appear at the reduced scale. This scaling gap was the single largest underestimation in the original plan.

9. Project Reflections

The following section reflects on the project outcomes, the gap between the original blueprint and the final delivered system, and the lessons learned during development. It covers the team's assessment of which design changes were deliberate engineering tradeoffs versus reactive decisions made under schedule pressure, and what the team would do differently if starting over.

9.1 Team-level Reflection

The honest assessment of this project is that the team built something that works, but not the thing the team originally set out to build. The blueprint described a fully self-contained FPGA accelerator capable of running an entire transformer inference pass in programmable logic, complete with on-chip softmax, layer normalization, and

GELU. The final system utilizes the Scheduler FSM as an instruction-level dispatcher to accelerate matrix multiplications on the FPGA, while the ARM CPU manages the high-level transformer sequence and non-linear math. The model is GPT-2 instead of Phi-3 Mini. The quantization is INT8 instead of INT4. The FPGA acts as a firmware-driven compute engine rather than a fully autonomous inference accelerator.

Some of these changes were good engineering decisions made for the right reasons. Choosing tiling over systolic arrays gave the team a debuggable compute architecture that could be verified module by module. Switching from Phi-3 to GPT-2 eliminated hardware complexity (CORDIC units for RoPE, sigmoid for SwiGLU) that the FPGA fabric could not afford and freed DDR4 headroom. These were deliberate tradeoffs, documented in the design process and made before the schedule was under pressure.

Other changes were reactive. The March 13 pivot happened because the team ran out of time to debug full-scale integration, not because the architecture was wrong in principle. The reduced-dimension models simulated correctly. The individual modules passed their verification tests. But assembling them into a working system at full scale, with real AXI transactions, real DDR bandwidth constraints, and real timing closure, exposed issues that simulation had not surfaced. The team had planned for this possibility (the blueprint's P7 mitigation strategy explicitly described the MAC-only fallback), but relying on the fallback was still a disappointment.

If the team were starting over, two things would change. First, integration testing on actual hardware would begin earlier and at intermediate scales (hidden dimension 128, then 512, then full). The gap between those two extremes hid problems that only became visible under pressure. Second, the mock AXI interfaces used during module development would be replaced with cycle-accurate AXI models from the start. The time saved by using simplified mocks during development was lost entirely during the integration phase when every module needed reworking to handle real protocol behaviour.

9.2 Broader Lessons

The project reinforced an important lesson in hardware design, simulation is not hardware. Every module in the LiteLM accelerator passed its C-simulation tests. The Scheduler FSM, MMU, and Compute Controller each produced correct outputs for their respective test vectors. The system failed when those modules had to operate together at full scale on physical hardware under real timing constraints. This is not a new lesson in hardware-software co-design, but experiencing it firsthand made the principal concrete in a way that coursework had not.

The other lesson is about scope. The blueprint was written with the ambition appropriate to a capstone project, but the team underestimated the integration effort for FPGA-based systems. In hindsight, scoping the project to a MAC-only accelerator from the start, with on-chip non-linear operations as an explicit stretch goal, would have produced a cleaner result and reduced the pressure that led to the March pivot. The work done on the full accelerator architecture was not wasted (the Scheduler FSM, MMU, and CC are complete, verified designs), but it did not ship as originally intended.

References

- [1] C. L. Zavazava, "Facts and Figures 2025 Foreword," 2025. [Online]. Available: <https://www.itu.int/itu-d/reports/statistics/2025/10/15/ff25-foreword/>.
- [2] "GPT-2," 16 11 2020. [Online]. Available: https://huggingface.co/docs/transformers/model_doc/gpt2. [Accessed 10 2025].
- [3] "phi3," 24 04 2024. [Online]. Available: https://huggingface.co/docs/transformers/model_doc/phi3 . [Accessed 10 2025].
- [4] M. Bekmyrza, "Language Model Inference on FPGA with Integer-only Operations," University of Waterloo, Waterloo, Ontario, 2025.
- [5] "Sustainable Development Goal 4: Quality Education," [Online]. Available: <https://www.un.org/sustainabledevelopment/education/>.
- [6] M. Kuhfeld, "The pandemic has had devastating impacts on learning. What will it take to help students catch up?," 3 March 2022. [Online]. Available: <https://www.brookings.edu/articles/the-pandemic-has-had-devastating-impacts-on-learning-what-will-it-take-to-help-students-catch-up/>.
- [7] AMD, "Zynq UltraScale+ Device," 21 March 2025. [Online]. Available: <https://docs.amd.com/v/u/en-US/ug1085-zynq-ultrascale-trm>. [Accessed 26 October 2025].
- [8] AMD, "AMBA® AXI4 Interface Protocol," AMD, 2025. [Online]. Available: <https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/axi.html#tabs-be33810108-item-3726c2a412-tab>. [Accessed 7 October 2025].
- [9] ARM, "ARM - AMBA 4," ARM, 2025. [Online]. Available: <https://www.arm.com/architecture/system-architectures/amba/amba-4>. [Accessed 7 October 2025].
- [10] "Kria K26 SOM Data Sheet (DS987)," 25 March 2026. [Online]. Available: <https://docs.amd.com/r/en-US/ds987-k26-som/Overview>.
- [11] "NVIDIA A100 Tensor Core GPU," 2026. [Online]. Available: <https://www.nvidia.com/en-us/data-center/a100/>.
- [12] "API Pricing," 2026. [Online]. Available: <https://openai.com/api/pricing/>.
- [13] "C-27," 2025. [Online]. Available: <https://www.parl.ca/legisinfo/en/bill/44-1/c-27>.
- [14] "Government of Canada," September 2023. [Online]. Available: <https://ised-isde.canada.ca/site/ised/en/voluntary-code-conduct-responsible-development-and-management-advanced-generative-ai-systems>.
- [15] "Professional Engineers Ontario Code of Ethics Ontario Regulation 941, section 77," 30 March 2026. [Online].
- [16] "Practice Advisory: Use of Artificial Intelligence in Professional Practice," 22 November 2024. [Online]. Available: https://www.peo.on.ca/sites/default/files/2025-11/EGBC_Use_of_Artificial_Intelligence_in_Professional_Work.pdf.

Appendix

Glossary

Address Generation Unit (AGU): Hardware that computes memory addresses for tiled/banked accesses so multiple BRAM/URAM banks can be read/written in parallel without conflicts.

AXI Protocol Checker IP: Vivado IP that verifies AXI4-Lite/AXI4-Full/AXI4-Stream timing and handshake rules during simulation or on-board debug, used to validate PS↔PL links.

Burst (AXI / DMA Burst): A sequence of back-to-back AXI data beats issued by the DMA engine to raise effective DDR throughput (e.g., for weight/KV tiles). Your datapath uses DMA bursts when pulling KV cache segments.

Controller FSM / Layer Sequencer: Control logic that orders per-layer operations (DMA loads, MAC triggers, normalization, status signaling) and advances decode steps.

Control/Status Register (CSR): Memory-mapped register (via AXI4-Lite) for configuration (e.g., start/stop, scales) and read-back of status/flags.

Cosine Similarity (embedding check): A metric used to confirm quantized embeddings remain close to FP32 reference; your test gate uses ≥ 0.98 .

Detokenization (Detok): Converting selected token IDs back into UTF-8 text for display (HDMI/UART).

GEMM (General Matrix - Matrix Multiply): The core matrix multiply used across projections/MLP; your Milestone 3 calls out INT8/INT4 GEMM kernels integrated with PS.

Guardrails: Output filters (e.g., banned-token regex, no-repeat-ngram) applied during sampling to keep generations coherent and safe.

Head Concatenation: The step that concatenates per-head attention outputs before the output projection.

I²C / SPI / GPIO: Low-speed peripheral interfaces available for debug/IO on the KV260 platform.

Initiation Interval (II): HLS pipeline metric indicating how often a loop can launch a new iteration (lower is better; II=1 is ideal).

KL Divergence (Softmax check): Distribution-level error metric comparing PL Softmax vs FP32 golden; used in your validation plan narrative.

Knowledge Pack / Model Pack: A versioned bundle (weights, scales/zero-points, tokenizer, sampler config) that can be loaded from storage into DDR, enabling offline “domain swaps.”

Latency P95 (P95): The 95th-percentile latency metric; your acceptance gate uses TTFT ≤ 300 ms median and ≤ 500 ms P95.

Logits: Raw (pre-Softmax) model scores that the PS sampler consumes to pick the next token; streamed PL→PS via AXI-Stream.

MMCM / PLL (Clock Gen): Clocking resources to generate, multiply, or phase-align clocks; typically paired with CDC logic at PL boundaries.

No-Repeat-N-gram: Decoding constraint that prevents repeating n-grams (e.g., n=2). It’s in your Sampler & Guardrails acceptance tests.

ONNX Q/DQ (Quant/Dequant Graph): Export format embedding quantization and dequantization ops plus sidecar scales/zero-points for hardware-friendly deployment.

Piecewise-Linear (PWL) Softmax / LUT Softmax: Fixed-point Softmax approximations using LUTs

or linear segments to save resources and latency.

Ping-Pong Buffering: Double-buffering trick (A/B) so one tile streams to MAC while the next tile prefetches from DDR, hiding DMA jitter.

Post-Training Quantization (PTQ) / QAT: PTQ calibrates an FP32 model with data to derive scales; QAT trains with fake-quant ops to recover accuracy if PTQ degrades quality.

Re-Quantization (int16→int8): Step that clamps/rounds accumulators to int8 at module boundaries (with per-tensor/group scales and zero-points) to keep data on-chip efficient.

ROUGE-L: Sequence-overlap metric used in some E2E parity checks (long-form text quality).

Sampler (Top-k / Top-p): Probabilistic decoding that truncates the candidate set by top-k or top-p (nucleus) before sampling the next token; part of your A4 gate.

Skid Buffer (AXI-Stream): Small FIFO/register slice that absorbs valid/ready bubbles and prevents protocol violations under backpressure.

Soak Test: Long-duration stability run (e.g., 8 hours) logging token rate, TTFT, temps, and AXI errors to verify there are no leaks or underruns.

Tokenization: Converting input text to token IDs (PS-side) prior to embedding; your pipeline streams tokens via AXI-Stream into PL.

Top-Level PL Shell: Structural wrapper that instantiates all PL modules, wires clocks/reset, and exposes AXI4-Lite/FULL/Stream to PS.

WNS / TNS (Timing): Worst Negative Slack (WNS) and Total Negative Slack (TNS) - key STA sign-off metrics in your acceptance table; target $WNS \geq 0$ ns with headroom.

Relevant Figure

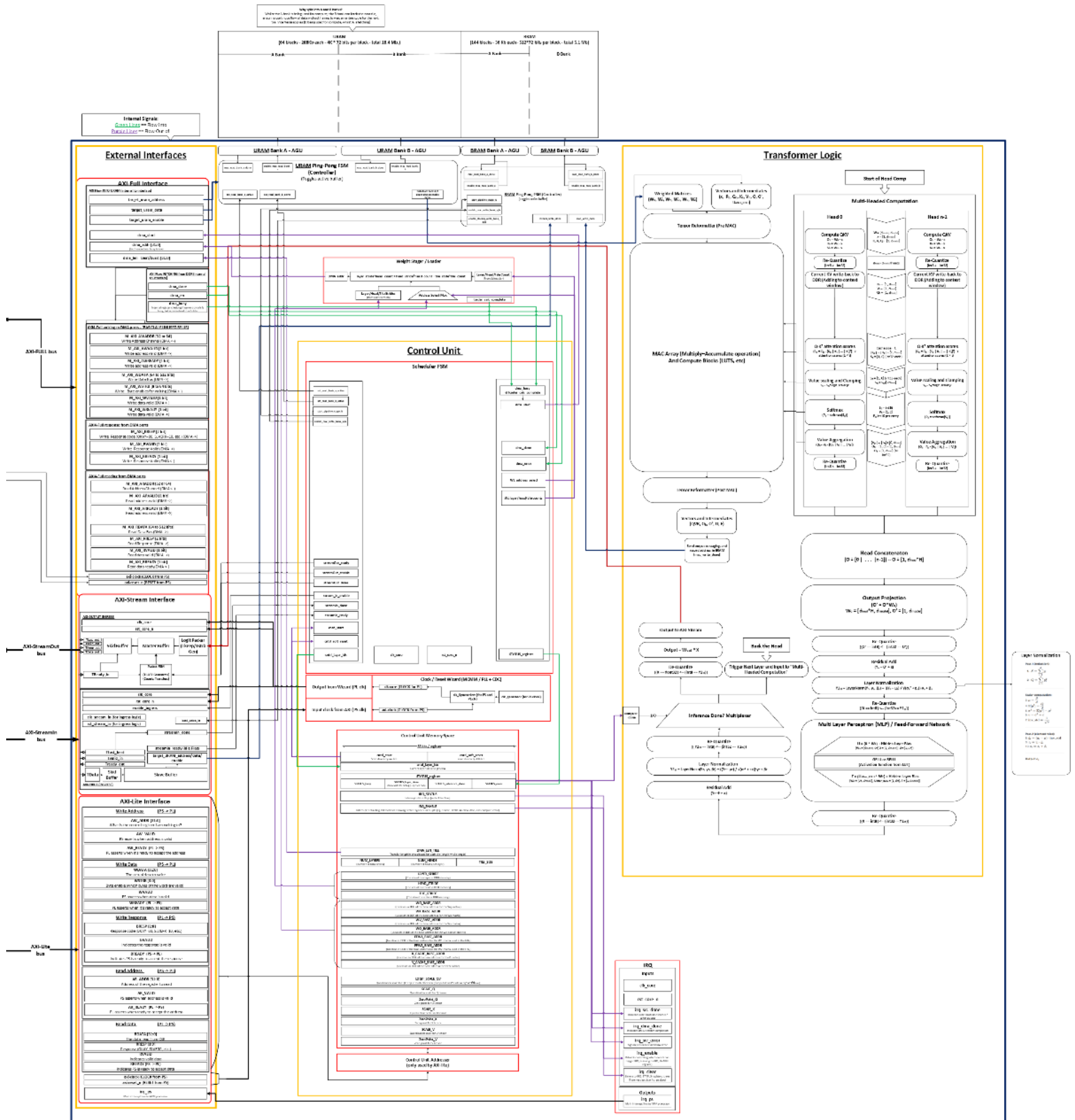


Figure 1: Complete system block diagram showing all module-level signal connections between the Scheduler FSM, MMU, Compute Controller, AXI, and DDR memory

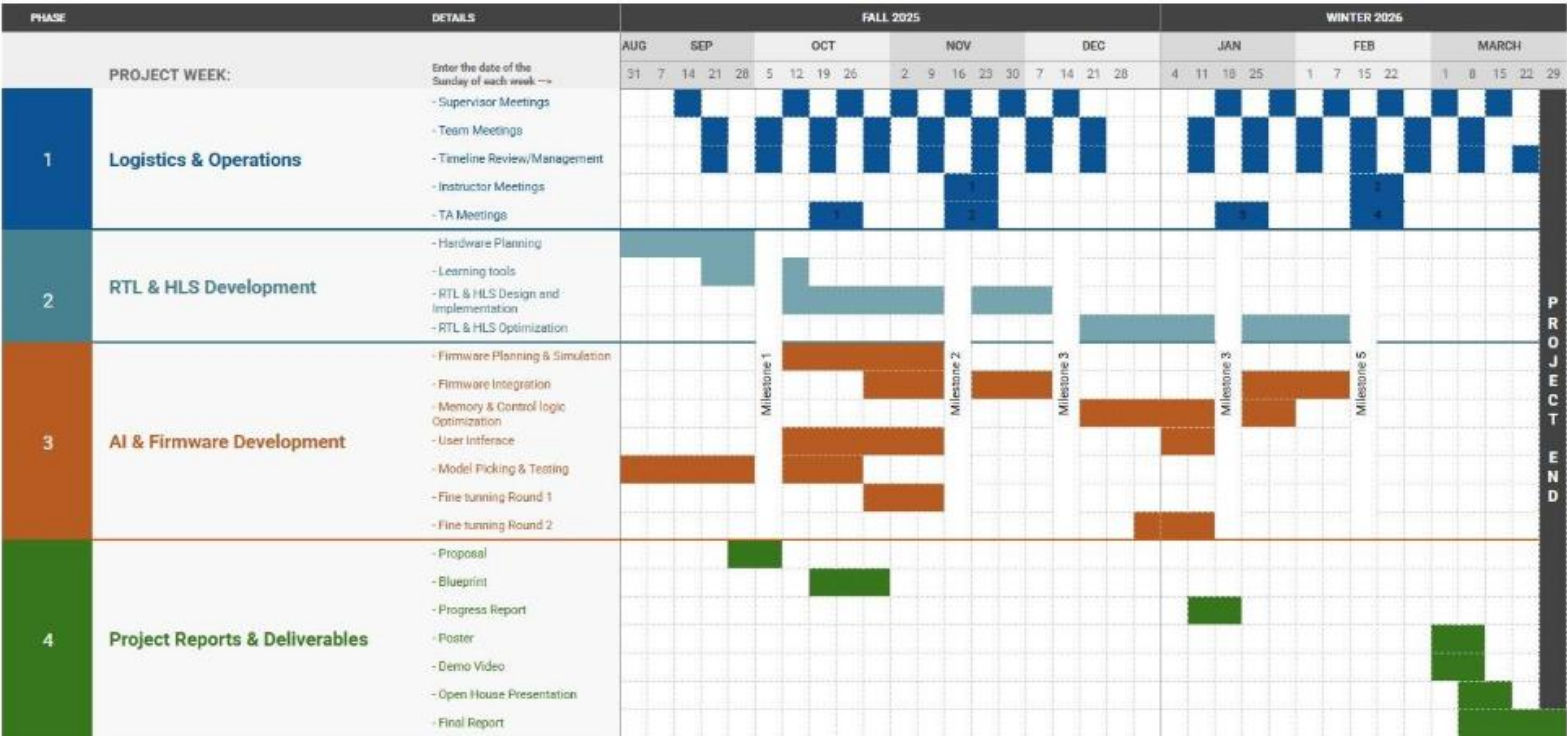


Figure 2: Gantt chart timeline from Oct 5th 2025 to March 30th 2026

Member Contributions

Table 9: Table detailing member contribution towards project completion

Luka Rodrigues	100
Andy Wang	100
Tristan Alderson	100
Vivek Chokkalingam	100

Acknowledgments

Table 10: Table detailing individuals that provided support for the project

Name	Role
Professor Ryan Grant	Project Supervisor
Heidi Miller	Course Instructor
Curtis Shorts	Course TA

We want to thank everyone for their help throughout the course of the project. It would not have been possible without their support.